Project 3 Report

Design

본 과제의 목표는 xv6 운영체제에 메모리 맵 파일(Memory-Mapped Files) 기능을 지원하는 `mmap`과 `munmap` 시스템 콜을 구현하는 것이다. 전체 파일 데이터를 미리 메모리에 올리지 않고, Lazy Allocation을 통해 페이지 폴트가 발생했을 때만 데이터를 메모리에 적재하도록 설계하였다.

과제 요구사항을 충족하기 위한 전체적인 구현 계획은 다음과 같다.

1. **Per-process VMA Bookkeeping**: 프로세스마다 최대 16개(`NVMA`)의 메모리 매핑 정보를 추적할 수 있도록 `struct vma`를 사용한다. `sys_mmap` 호출 시 사용되지 않는 VMA 슬롯을 찾아 매핑 정보를 기록하고 파일의 참조 횟수(`filedup`)를 증가시킨다.
2. **Lazy Page-Fault Handler**: `mmapfault` 트랩 핸들러를 구현하여, 매핑된 영역에 최초로 접근할 때 발생하는 페이지 폴트를 처리한다. 이때 물리 메모리(`kalloc`)를 할당하고, 매핑된 파일의 지정된 오프셋에서 데이터를 읽어와(`readi`) 페이지 테이블에 매핑(`mappages`)한다.
3. **munmap & Write-back**: `sys_munmap` 호출 시 매핑된 메모리를 해제(`uvmunmap`)한다. 만약 `MAP_SHARED` 플래그로 매핑된 영역이라면, 메모리 해제 전 디스크의 원본 파일에 변경 사항을 다시 기록(`writei`)한다.
4. **kfork & kexit Handling**: 자식 프로세스 생성(`kfork`) 시 부모의 VMA 배열을 복사하여 동일한 매핑 정보를 공유하도록 한다. 프로세스 종료(`kexit`) 시에는 사용 중인 모든 VMA 영역에 대해 `munmap`과 동일하게 `MAP_SHARED` 영역을 write-back 처리하고 자원을 정리한다.

Implementation

주어진 xv6 스켈레톤 코드에서 `kernel/sysfile.c`와 `kernel/proc.c`를 중심으로 다음 내용들을 구현 및 수정하였다.

헤더 파일 의존성 해결 (`kernel/proc.c`, `kernel/sysfile.c`)

VMA 내의 `struct file` 및 inode에 접근하기 위해 헤더를 추가하였다. 헤더 참조 순서에 의한 `incomplete type` 오류를 방지하기 위해 `sleeplock.h`, `fs.h`, `file.h` 순서로 안전하게 include 하였으며, `TRAPFRAME` 상수를 사용하기 위해 `memlayout.h`를 포함하였다.

`sys_mmap` (`kernel/sysfile.c`)

```
uint64
sys_mmap(void)
{
  int len, prot, flags, fd, offset;
  struct file *f;

  argint(0, &len);
  argint(1, &prot);
  argint(2, &flags);
  if(argfd(3, &fd, &f) < 0)
    return -1;
  argint(4, &offset);

  if(len <= 0 || (offset % PGSIZE) != 0)
    return -1;

  if(!f->readable && (prot & PROT_READ)) return -1;
  if(!f->writable && (prot & PROT_WRITE) && (flags == MAP_S
HARED)) return -1;

  struct proc *p = myproc();
  struct vma *v = 0;

  for(int i = 0; i < NVMA; i++) {
    if(p->vmas[i].used == 0) {
      v = &p->vmas[i];
      break;
    }
  }
  if(!v) return -1;

  uint64 map_start = TRAPFRAME;
  for(int i = 0; i < NVMA; i++){
```

```
    if(p->vmas[i].used && p->vmas[i].addr < map_start) {
      map_start = p->vmas[i].addr;
    }
  }
  map_start -= PGROUNDUP(len);
  if(map_start < p->sz) return -1;

  v->used = 1;
  v->addr = map_start;
  v->length = len;
  v->prot = prot;
  v->flags = flags;
  v->offset = offset;
  v->f = filedup(f);

  return v->addr;
}
```

- `argint`, `argfd`를 사용해 인자를 파싱한다. `argint`는 void를 반환하므로 리턴값 검사 없이 변수에 값을 담도록 구현했다.
- 파일 권한과 `mmap`의 `prot` 플래그 간의 충돌(`PROT_WRITE` 요청 시 파일이 쓰기 불가능한 경우 등)을 검사한다.
- 현재 프로세스의 VMA 배열을 순회하며 빈 슬롯을 찾는다.
- `TRAPFRAME` 아래부터 힙 영역(`p->sz`) 위쪽 공간 사이에서, 다른 VMA와 겹치지 않는 페이지 정렬된 가상 주소(`VA`) 공간을 할당받아 VMA 구조체를 초기화하고 반환한다.

`mmapfault` (`kernel/sysfile.c`)

```
int
mmapfault(uint64 va)
{
  struct proc *p = myproc();
  struct vma *v = 0;

  for(int i = 0; i < NVMA; i++) {
    if(p->vmas[i].used && va >= p->vmas[i].addr && va < p->
vmas[i].addr + p->vmas[i].length) {
```

```
      v = &p->vmas[i];
      break;
    }
  }
  if(!v) return -1;

  uint64 va_align = PGROUNDDOWN(va);
  if(walkaddr(p->pagetable, va_align) != 0) return -1;

  char *mem = kalloc();
  if(!mem) return -1;
  memset(mem, 0, PGSIZE);

  ilock(v->f->ip);
  int read_offset = v->offset + (va_align - v->addr);
  readi(v->f->ip, 0, (uint64)mem, read_offset, PGSIZE);
  iunlock(v->f->ip);

  int perm = PTE_U;
  if(v->prot & PROT_READ) perm |= PTE_R;
  if(v->prot & PROT_WRITE) perm |= PTE_W;

  if(mappages(p->pagetable, va_align, PGSIZE, (uint64)mem,
perm) < 0) {
    kfree(mem);
    return -1;
  }

  return 0;
}
```

- 페이지 폴트가 발생한 가상 주소(`va`)를 포함하는 VMA를 찾는다.
- 이미 매핑된 페이지에 대한 권한 폴트인지 확인한 후, 아니라면 `kalloc()`으로 1페이지(PGSIZE) 크기의 물리 메모리를 할당받아 0으로 초기화한다.
- `v->f->ip`의 락을 획득한 후 `readi` 함수를 호출하여, 파일의 데이터를 할당된 물리 메모리로 읽어온다.

- `PROT_READ`, `PROT_WRITE` 플래그를 xv6의 `PTE_R`, `PTE_W`, `PTE_U` 플래그로 변환하여 `mappages`로 프로세스의 페이지 테이블에 매핑한다.

`sys_munmap` (`kernel/sysfile.c`)

```
uint64
sys_munmap(void)
{
  uint64 addr;
  int length;

  argaddr(0, &addr);
  argint(1, &length);

  struct proc *p = myproc();
  struct vma *v = 0;

  for(int i = 0; i < NVMA; i++) {
    if(p->vmas[i].used && addr >= p->vmas[i].addr && addr <
p->vmas[i].addr + p->vmas[i].length) {
      v = &p->vmas[i];
      break;
    }
  }
  if(!v) return -1;

  if(addr != v->addr && addr + length != v->addr + v->lengt
h)
    return -1;

  uint64 va = addr;
  uint64 end = addr + length;

  while(va < end) {
    uint64 pa = walkaddr(p->pagetable, va);
    if(pa != 0) {
      if(v->flags == MAP_SHARED) {
        begin_op();
        ilock(v->f->ip);
```

```
        writei(v->f->ip, 0, pa, v->offset + (va - v->addr),
PGSIZE);
        iunlock(v->f->ip);
        end_op();
      }
      uvmunmap(p->pagetable, va, 1, 1);
    }
    va += PGSIZE;
  }

  if(addr == v->addr && length == v->length) {
    fileclose(v->f);
    v->used = 0;
  } else if(addr == v->addr) {
    v->addr += length;
    v->length -= length;
    v->offset += length;
  } else {
    v->length -= length;
  }

  return 0;
}
```

- 해제할 범위를 포함하는 VMA를 찾고, 중간 영역만 해제하는 잘못된 요청인지 검증한다.
- 해당 범위의 가상 주소를 순회하며 실제로 할당되어 있는 페이지(`walkaddr`로 검사)를 찾는다.
- `MAP_SHARED`인 경우 `writei`를 호출하여 메모리의 데이터를 파일에 동기화(write-back)한다.
- `uvmunmap`으로 페이지 테이블 매핑을 해제하고 물리 메모리를 반환한다. 영역이 완전히 해제된 경우 `fileclose(v->f)`를 호출하고 VMA 슬롯을 비운다.

`kfork` 및 `kexit` (`kernel/proc.c`)

```
int
kfork(void)
```

```
{
    ...
    np->sz = p->sz;

  for(int i = 0; i < NVMA; i++) {
    np->vmas[i] = p->vmas[i];
    if(np->vmas[i].used) {
      filedup(np->vmas[i].f);
    }
  }
  ...
```

```
void
kexit(int status)
{
  struct proc *p = myproc();

  if(p == initproc)
    panic("init exiting");
  for(int i = 0; i < NVMA; i++) {
    if(p->vmas[i].used) {
      struct vma *v = &p->vmas[i];
      for(uint64 va = v->addr; va < v->addr + v->length; va
+= PGSIZE) {
        uint64 pa = walkaddr(p->pagetable, va);
        if(pa != 0) {
          if(v->flags == MAP_SHARED) {
            begin_op();
            ilock(v->f->ip);
            writei(v->f->ip, 0, pa, v->offset + (va - v->ad
dr), PGSIZE);
            iunlock(v->f->ip);
            end_op();
          }
          uvmunmap(p->pagetable, va, 1, 1);
        }
      }
      fileclose(v->f);
```

```
        v->used = 0;
      }
    }
    ...
```

- `kfork()`: 부모 프로세스의 메모리를 복사한 직후, `p->vmas` 배열을 자식 `np->vmas`로 복사한다. 이때 `used` 상태인 VMA의 파일에 대해 `filedup()`을 호출하여 파일 참조 횟수를 증가시켰다. 물리 페이지는 Lazy allocation되므로 페이지 테이블 매핑은 복사하지 않았다.
- `kexit()`: 프로세스가 종료되어 열린 파일을 모두 닫기 전에, 활성화된 모든 VMA를 검사한다. 매핑된 영역 중 `MAP_SHARED`로 설정된 실제 물리 페이지 데이터를 찾아 디스크 파일에 write-back(`writei`)하고 매핑을 해제하였다.

Results

구현을 마친 후 `make qemu` 환경에서 과제 명세의 검증 도구인 `mmaptest`와 `usertests -q`를 실행하여 구현의 완전성을 확인하였다.

- `mmaptest` 실행 시 모든 단일 프로세스 매핑 테스트, 파일 읽기/쓰기 동기화 테스트, 그리고 fork를 통한 부모-자식 간 매핑 공유 테스트가 문제없이 성공하였다.
- `usertests -q` 실행 시 기존 xv6 시스템 콜 동작에 부작용을 일으키지 않고 모든 테스트 케이스를 통과하는 것을 확인하였다.

```
$ mmaptest
test basic mmap
test basic mmap: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
test fork: OK
test munmap prevents access
usertrap(): unexpected scause 0xd pid=5
            sepc=0xaae stval=0x3fffffd000
usertrap(): unexpected scause 0xd pid=6
            sepc=0xb28 stval=0x3fffffc000
test munmap prevents access: OK
test writes to read-only mapped memory
usertrap(): unexpected scause 0xf pid=7
            sepc=0xc80 stval=0x3fffffc000
test writes to read-only mapped memory: OK
mmaptest: all tests succeeded
```

```
usertrap(): unexpected scause 0xf pid=6646
            sepc=0x46d8 stval=0x3f012000
PASS
test lazy_copy: PASS
test lazy_sbrk: PASS
PASS ALL TESTS
```

위 실행 흐름을 통해 Lazy Loading이 정상 동작하며, 프로세스 종료 시 `MAP_SHARED` 영역에 대한 write-back 및 파일 리소스 반환이 메모리 누수 없이 완벽히 수행됨을 확인할 수 있었다.

Troubleshooting

과제 진행 중 다음과 같은 컴파일 오류 및 논리적 이슈를 마주하였고, 이를 성공적으로 해결하였다.

1. **Forward Declaration 및 의존성 오류**
    - **문제**: `kernel/proc.c`에서 `v->f->ip`에 접근하려 할 때 `invalid use of undefined type 'struct file'` 에러 및 `incomplete type 'struct sleeplock'` 에러가 연쇄적으로 발생하였다.
    - **해결**: xv6의 파일 시스템 관련 구조체(`file`, `inode`)는 내부에 `sleeplock`을 포함하므로 헤더 파일의 include 순서가 매우 중요함을 파악했다. `sleeplock.h` → `fs.h` → `file.h` 순서로 의존성에 맞게 헤더를 배치하여 구조체 미정의 에러를 해결하였다.
2. **시스템 콜 인자 반환값 검사(void value not ignored) 오류**
    - **문제**: `sys_mmap` 및 `sys_munmap`에서 `if(argint(...) < 0)` 형태로 인자를 파싱할 때 컴파일 에러가 발생했다.
    - **해결**: 제공된 스켈레톤 코드의 `syscall.h` 및 커널 환경에서 `argint`, `argaddr` 함수가 `void` 타입으로 선언되어 있음을 확인했다. 조건문 안에서 반환값을 검사하던 로직을 분리하여 단순히 인자를 읽어오는 형태로 코드를 수정하여 해결하였다.
      단, `argfd`는 `int` 반환 타입이므로 정상적으로 예외 처리 로직을 유지하였다.
3. **TRAPFRAME 누락 오류**
    - **문제**: mmap 할당 주소 계산 시 `TRAPFRAME` 상수를 찾지 못하는 오류가 발생했다.
    - **해결**: 해당 상수가 정의되어 있는 `memlayout.h`를 파일 최상단에 include 하여 해결하였다.